

NetTower Developer Documentation

Developer Documentation

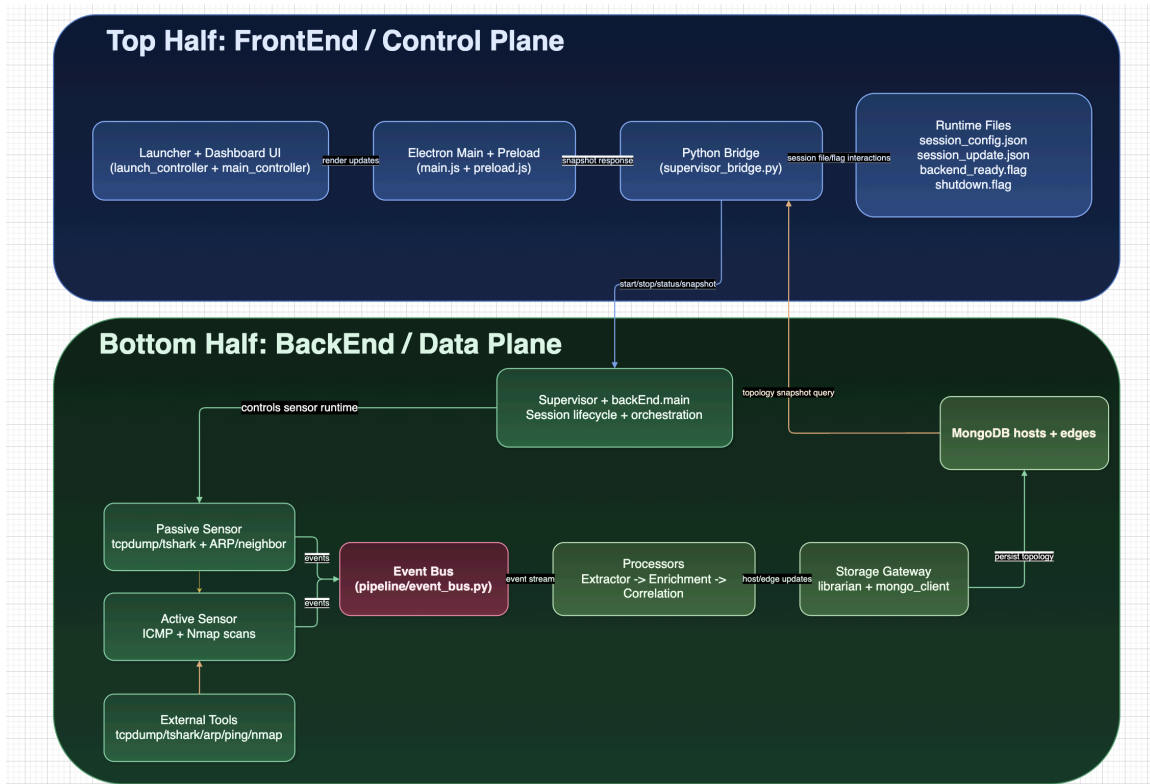


Table of Contents

1. Audience and Goals
2. Project Overview
3. Architecture at a Glance
 - 3.1 Process Boundaries
 - 3.2 Data Boundaries
4. Repository Layout
5. Prerequisites and Tooling
6. Environment Setup (Fresh Machine)

- 6.1 Python Environment
- 6.2 Frontend Environment
- 6.3 Network Tool Verification
- 7. Running the Application End-to-End
- 8. Runtime Lifecycle and IPC Contract
 - 8.1 Electron IPC Surface
 - 8.2 Bridge Protocol
- 9. Backend Pipeline Deep Dive
- 10. Passive Discovery Internals
- 11. Active Discovery Internals
- 12. Correlation and Topology Inference
- 13. Storage Model and Mongo Contract
 - 13.1 Hosts Collection
 - 13.2 Edges Collection
 - 13.3 Indexes
- 14. Frontend Architecture
 - 14.1 Electron Main Process (`frontEnd/main.js``)
 - 14.2 Launch Renderer (`launch_controller.js``)
 - 14.3 Main Renderer (`main_controller.js``)
- 15. Topology Visualization Engine
- 16. Runtime Settings and Dynamic Session Updates
- 17. Runtime Files and Flags
- 18. Testing Strategy
- 19. Debugging and Troubleshooting Playbook
 - 19.1 Session Startup Failures
 - 19.2 No Topology Updates

19.3 Nmap Warnings

19.4 Edge/Role Inference Anomalies

20. Common Development Tasks

21. Extending the Frontend Safely

22. Security and Scan Safety

23. Known Gaps and Future Work

24. Contributor Checklist

25. Appendix: Quick Reference

Document purpose: this guide is for developers joining NetTower with no prior project context.

Version scope: current repository state under `/code` with Python backend, Electron frontend, and Mongo-backed topology runtime.

1. Audience and Goals

This document is for:

- Developers implementing new features in backend or frontend.
- Engineers debugging runtime/session/scan behavior.
- Contributors extending topology inference, visualization, or test coverage.

Primary goals:

- Explain the architecture from process boundary to data model.
- Explain the runtime lifecycle and where state flows.
- Explain how to safely extend discovery and inference logic.
- Explain how to run, test, and debug the project quickly.

2. Project Overview

NetTower is an agentless network discovery and topology application.

The system has three major layers:

- Electron desktop UI (`frontEnd`) for session control and topology rendering.
- Python runtime supervisor + backend pipeline (`backEnd`) for discovery, inference, and persistence.
- Runtime filesystem + Mongo process (`runtime` and `runtime\_binaries`) for shared state and data storage.

Core behavior:

- User configures a session in the launcher.
- Electron starts a Python bridge.
- Python bridge drives `Supervisor`.
- Supervisor starts Mongo + backend.
- Backend writes `runtime/backend\_ready.flag`.
- Frontend polls topology snapshots from Mongo through the bridge.
- User can adjust runtime active-scan settings while session is running.

3. Architecture at a Glance

3.1 Process Boundaries

Electron main process:

- Owns window lifecycle and IPC handlers.
- Spawns `frontEnd/supervisor\_bridge.py`.
- Never reads Mongo directly.

Electron renderer processes:

- ``launch.html`` + ``launch_controller.js`` for startup config.
- ``main.html`` + ``main_controller.js`` for topology dashboard.
- Talks only through preload APIs (``window.nettower.*``).

Python bridge (``frontEnd/supervisor_bridge.py``):

- JSON-line protocol over stdin/stdout.
- Owns one ``Supervisor`` instance.
- Handles commands: ``start_session``, ``stop_session``, ``status``, ``get_topology_snapshot``, ``shutdown``.

Supervisor/backend:

- ``Supervisor`` controls Mongo lifecycle and backend process lifecycle.
- Backend pipeline (``backEnd/main.py``) runs sensors/processors and persists host/edge entities.

3.2 Data Boundaries

Runtime files:

- ``runtime/session_config.json``: session config used by backend startup.
- ``runtime/session_update.json``: runtime toggle updates written by frontend.
- ``runtime/backend_ready.flag``: readiness signal created by backend.
- ``runtime/shutdown.flag``: graceful shutdown request signal.

Mongo collections:

- ``hosts``: canonical host entities.
- ``edges``: canonical edge entities.

4. Repository Layout

Top-level directories:

- ``backEnd/``: Python runtime, sensors, processors, models, storage.
- ``frontEnd/``: Electron app, HTML/CSS/JS renderer, Python bridge.
- ``runtime/``: shared runtime files and temporary session artifacts.
- ``runtime_binaries/``: bundled runtime binaries (Mongo and tshark where available).
- ``tests/``: automated tests and custom test runner.

Important frontend files:

- ``frontEnd/main.js``: Electron main process, bridge process manager, IPC.
- ``frontEnd/preload.js``: secure API surface for renderer.
- ``frontEnd/windows/launch.html``: launch/config UI.
- ``frontEnd/windows/main.html``: dashboard UI.
- ``frontEnd/js/launch_controller.js``: startup validation + start action.
- ``frontEnd/js/main_controller.js``: polling, rendering, interactions, settings.
- ``frontEnd/supervisor_bridge.py``: Python command bridge.

Important backend files:

- ``backEnd/main.py``: backend event loop entrypoint.
- ``backEnd/runtime/supervisor.py``: session runtime orchestration.
- ``backEnd/sensors/passive_listener.py``: passive packet observation.
- ``backEnd/sensors/active_discovery.py``: active discovery and nmap integration.
- ``backEnd/processors/extractors.py``: raw-to-event normalization.
- ``backEnd/processors/enrichment.py``: vendor/role/os hints.
- ``backEnd/processors/correlation.py``: inference + host/edge correlation.
- ``backEnd/storage/mongo_client.py``: Mongo connection and indexes.
- ``backEnd/storage/librarian.py``: storage gateway for host/edge reads/writes.

5. Prerequisites and Tooling

Required:

- Python 3.x (same interpreter used by Electron when spawning bridge).
- Node.js + npm for Electron frontend.
- Mongo binary in `runtime\_binaries` (or custom binary path via runtime config).
- `pymongo` Python package.
- `psutil` Python package.

Discovery tool dependencies:

- Passive capture: `tcpdump` preferred, `tshark` fallback.
- Active scanning: `ping` for ICMP checks, `nmap` for nmap scan paths.

Platform notes:

- Packet capture can require elevated privileges/capabilities.
- SYN scans (`-sS`) require elevated privileges on many systems; scanner auto-falls back to `-sT`.
- If `nmap` is not installed or not in path/common locations, nmap steps are skipped.

6. Environment Setup (Fresh Machine)

6.1 Python Environment

From project root:

- `python3 -m venv venv`
- `source venv/bin/activate` (Windows: `venv\Scripts\activate`)
- Install required packages used by code imports:
- `pip install pymongo psutil`

If you run tests:

- Ensure the same interpreter resolves project modules from root.

6.2 Frontend Environment

From ``frontEnd/``:

- ``npm install``
- ``npm start``

``package.json`` scripts:

- ``start``: runs ``electron .``
- ``dev``: same as start

6.3 Network Tool Verification

Before active scanning work:

- Verify ``ping`` availability.
- Verify ``nmap`` availability from the same shell context used to launch app.

Before passive scanning work:

- Verify ``tcpdump`` or ``tshark`` availability.
- Confirm capture permissions for selected interface.

7. Running the Application End-to-End

1. Start frontend:

- ``cd frontEnd``
- ``npm start``

2. Fill launcher form:

- Interface (required)
- Discovery CIDR (required)
- Passive toggle
- Active toggle
- Discovery interval
- Targeted cooldown

3. Press Start Session:

- Electron asks bridge to start session.
- Bridge validates config via ``SessionConfig.from_dict(...).validate()``.
- Supervisor starts Mongo and backend.
- Backend clears collections and writes readiness flag.
- UI transitions to dashboard.

4. Stop Session from dashboard:

- Renderer invokes ``runtime:stop-session``.
- Bridge calls ``Supervisor.stop_session()``.
- Runtime flags cleaned.
- UI returns to launcher.

8. Runtime Lifecycle and IPC Contract

8.1 Electron IPC Surface

Exposed in ``preload.js``:

- ``startSession(sessionConfig)``
- ``stopSession()``
- ``getStatus()``
- ``getTopology(options)``

- ``getLocalIdentity()``
- ``getSessionSettings()``
- ``updateSessionSettings(settingsPatch)``

Implemented in ``frontEnd/main.js`` IPC handlers:

- ``runtime:start-session``
- ``runtime:stop-session``
- ``runtime:get-status``
- ``runtime:get-topology``
- ``runtime:get-local-identity``
- ``runtime:get-session-settings``
- ``runtime:update-session-settings``

8.2 Bridge Protocol

Transport:

- stdin/stdout JSON-line envelopes.

Request shape:

- ``{"id": <int>, "command": "<cmd>", "payload": {...}}``

Response shape:

- success: ``{"id": <int>, "ok": true, "result": {...}}``
- error: ``{"id": <int>, "ok": false, "error": "<message>"}``

Bridge commands:

- ``start_session``
- ``stop_session``
- ``status``
- ``get_topology_snapshot``

- ``shutdown``

9. Backend Pipeline Deep Dive

Backend runtime entrypoint: ``backEnd/main.py``.

Main loop responsibilities:

- Load session config from runtime file.
- Detect local interface context (IP, MAC, default gateway, local network CIDR).
- Connect to Mongo and clear topology collections at start.
- Create event bus and subscribe main consumer.
- Start passive listener thread when enabled.
- Trigger active interval scans when enabled and due.
- Process incoming events through:
 - extractor
 - enricher
 - correlator
 - librarian upserts
- Schedule targeted active scans based on correlation signals.
- Watch shutdown flag for graceful stop.

Startup seeding:

- Local host and default gateway are seeded into topology early to anchor hierarchy.

10. Passive Discovery Internals

Implementation: ``backEnd/sensors/passive_listener.py``.

Backend selection:

- Prefer ``tcpdump``.
- Fallback to ``tshark``.
- If neither exists, passive listener disables itself with log warnings.

Captured signal types:

- IPv4 traffic tuple (``src_ip``, ``dst_ip``, ports/protocol).
- ARP request/reply patterns.
- MAC pair extraction from L2 headers.
- Packet length metadata.

Published events:

- ``host_seen`` from ARP and neighbor data.
- ``traffic_seen`` from IPv4 and ARP relationships.

Neighbor enrichment:

- Periodic ARP/neighbor table polling (``arp`` / ``ip neigh``) to sustain topology context.
- Emits host and local-to-neighbor ARP relationship events even during low packet volumes.

Failure behavior:

- If capture process exits unexpectedly, main loop logs and attempts controlled restart after cooldown.

11. Active Discovery Internals

Implementation: ``backEnd/sensors/active_discovery.py``.

Discovery modes:

- ICMP ping discovery.
- Nmap discovery/deep scans.
- Both can run together.

Run entry:

- ``run_discovery(cfg, bus, target, enable_icmp_scan, enable_nmap_scan)``
- ``target`` can be IP or CIDR.

Sweep pipeline (CIDR):

- Build host target list from CIDR.
- Optional ICMP ping fanout to identify alive hosts.
- Optional nmap discovery scan per target host.
- Select interesting hosts for deeper nmap profiling.

Targeted pipeline (IP):

- Optional ping.
- Optional nmap discovery.
- Deep nmap if evidence indicates host life or interest.

Nmap strategy in code:

- Discovery scan: focused role ports + host timeout + XML output.
- Deep scan phase:
- service/version scan (``-sV --version-light``)
- OS hints (``-O --osscan-limit``)
- traceroute (``--traceroute -sn``)

Life evidence gating:

- Hosts are emitted only when there is enough evidence (status reason, ports, MAC, etc.).
- This prevents polluting topology with dead/unassigned scanned IPs.

Nmap resolution:

- Uses ``shutil.which("nmap")`` and known fallback install paths.
- Logs warning once when missing and skips nmap operations.

12. Correlation and Topology Inference

Implementation: ``backEnd/processors/correlation.py``.

Core responsibilities:

- Merge event stream into stable ``host_entity`` and ``edge_entity`` updates.
- Infer host role, topology role, parent candidate, layer, external/internal class.
- Infer relationship types for edges.
- Emit signals for targeted active scans.

Relationship classes:

- ``upstream/external``
- ``gateway-for``
- ``routed-to``
- ``same-segment-peer``
- ``observed-traffic-peer``

Priority model:

- Stronger relationship labels override weaker ones on edge updates.

Parent/layer inference:

- External hosts: layer 0.
- Gateway host: layer 1.
- Router/switch tier: layer 2.

- Internal client/workstation tier: layer 3.
- Parent candidate usually points to inferred gateway/router when confidence exists.

Traceroute integration:

- ``route_hop_seen`` generates route edges and path chain edges.
- First hop strongly influences parent inference.
- Transit recurrence boosts router/network confidence for hop hosts.

Safety behavior:

- ``traffic_seen`` does not create destination host if only seen as outbound scan target without proof of life.

13. Storage Model and Mongo Contract

13.1 Hosts Collection

Canonical entity: ``host_entity`` in ``backEnd/models/entities.py``.

Key fields:

- identity: ``host_id``, ``ips``, ``macs``, ``hostnames``
- enrichment: ``vendor``, ``os_guess``
- role inference: ``role``, ``role_confidence``, ``role_scores``
- topology inference: ``node_role``, ``node_role_confidence``, ``parent_candidate``, ``parent_confidence``, ``topology_layer``, ``is_external``
- time: ``first_seen``, ``last_seen``
- active info: ``ports``, ``services``

13.2 Edges Collection

Canonical entity: ``edge_entity``.

Key fields:

- identity: ``edge_key``, ``a_host_id``, ``b_host_id``, ``proto``
- inference: ``relation``, ``relationship_type``, ``inferred``, ``confidence``, ``evidence``
- activity: ``first_seen``, ``last_seen``, ``count``, ``ports``

13.3 Indexes

Defined in ``backEnd/storage/mongo_client.py``.

Important indexes:

- hosts: ``host_id`` unique, plus ``ips``, ``macs``, ``last_seen``, ``role``, ``node_role``, ``parent_candidate``, ``topology_layer``
- edges: ``edge_key`` unique, ``(a_host_id,b_host_id,proto)``, ``last_seen``, ``relation``, ``relationship_type``

14. Frontend Architecture

14.1 Electron Main Process (``frontEnd/main.js``)

Responsibilities:

- Manage launch/main windows.
- Manage Python bridge process lifecycle.
- Normalize/validate session payload defaults.
- Handle IPC requests from renderer.
- Resolve local interface identity for local-host highlighting.

Bridge manager features:

- Python candidate resolution (`NETTOWER\_PYTHON`, venv paths, system python).
- Request/response command matching with timeouts.
- Captures bridge stderr tail for surfaced errors.
- Readiness waits on `runtime/backend\_ready.flag`.

14.2 Launch Renderer (`launch\_controller.js`)

Responsibilities:

- Gather launcher form values.
- Validate interface, CIDR, numeric bounds, and sensor mode selection.
- Show loading state while startup is in progress.
- Trigger `window.nettower.startSession`.

14.3 Main Renderer (`main\_controller.js`)

Responsibilities:

- Poll topology snapshots periodically.
- Normalize host/edge payloads into render state.
- Render 2D, 3D, Idle modes on canvas.
- Provide host filtering and stale-host visibility behavior.
- Show host detail drawer.
- Manage settings drawer and runtime setting updates.
- Provide text topology list in settings data tab.

15. Topology Visualization Engine

Canvas renderer supports:

- Mode rotation: `2d -> 3d -> idle`.
- Layout toggle: `tree` or `star`.
- Host filter toggle: `all`, `public`, `private`.
- Zoom slider with unified 2D/3D mapping.

Tree layout behavior:

- Builds hierarchical targets from inferred parent candidates and roles.
- Selects gateway candidate (local-edge relation and router confidence).
- Places external/public hosts in an adaptive top blob with relaxation pass to reduce overlap.
- In tree mode, public edges can be routed visually via gateway for readability.

2D mode:

- Physics-driven or tree-target interpolation depending on selected layout.
- Pan via pointer drag.

3D mode:

- Perspective projection with camera yaw/pitch/distance/pan.
- Wireframe ground grid.
- Node towers (3D cubes) and role glyph overlays.

Idle mode:

- Rotating presentation mode for passive display.

Local host marker:

- Drawn as a white rounded star around local host node in both 2D and 3D render paths.

16. Runtime Settings and Dynamic Session Updates

Source of truth:

- Frontend writes updates via ``runtime:update-session-settings``.
- Electron main writes ``runtime/session_update.json`` and merged ``session_config.json``.
- Backend loop applies updates via ``apply_runtime_session_updates(...)``.

Runtime-updatable toggles:

- ``enable_active_discovery``
- ``allow_all_active_targets``
- ``enable_icmp_scan``
- ``enable_nmap_scan``
- ``discovery_target_cidr`` (when provided)

Display-only frontend settings (not backend runtime state):

- stale host visibility toggles and threshold
- refresh interval
- host/edge render limits

17. Runtime Files and Flags

Shared runtime artifacts:

- ``runtime/backend_ready.flag``: backend readiness marker.
- ``runtime/shutdown.flag``: graceful shutdown request.
- ``runtime/session_config.json``: full normalized session state.
- ``runtime/session_update.json``: patch trigger consumed by backend loop.

Operational expectation:

- Ready flag is created after Mongo connect and collection reset.
- Ready/shutdown/session update files are cleaned aggressively on start/stop to avoid stale state.

18. Testing Strategy

Test runner:

- ``tests/run_code_tests.py``
- Discovers ``tests/test_*.py``.
- Prints evidence summary rows (name/result/pass-fail).

Current test coverage areas:

- session config validation and coercion
- enrichment hint behavior
- correlation role and topology inference behavior
- extractor conversion behavior
- snapshot response limit clamping
- runtime config normalization
- traffic-to-unknown destination guardrail behavior
- route-hop relationship and parent inference behavior

In-memory test double:

- ``tests/helpers.py`` provides ``InMemoryLibrarian`` to test correlator without Mongo.

Run tests from project root:

- ``python tests/run_code_tests.py``

19. Debugging and Troubleshooting Playbook

19.1 Session Startup Failures

Symptoms:

- ``Python bridge exited (code=1)``
- launch page shows start failure.

Check:

- Python interpreter resolution in ``frontEnd/main.js``.
- bridge stderr tail in Electron logs.
- required Python deps (``pymongo``, ``psutil``).
- Mongo binary availability in expected runtime paths.

19.2 No Topology Updates

Check:

- session running state (``runtime:get-status``).
- backend log for passive listener failures.
- interface selection correctness.
- packet capture permission.

19.3 Nmap Warnings

Symptom:

- ``nmap executable not found ... NMAP scans are skipped``.

Check:

- ``nmap`` available in shell path used to launch Electron.
- fallback install locations in active discovery resolver.

19.4 Edge/Role Inference Anomalies

Check:

- raw hosts/edges in Settings -> Data list.
- relationship labels and confidence values.
- parent_candidate and topology_layer fields for outliers.
- whether active sensor toggles are currently enabled.

20. Common Development Tasks

Add a new host role heuristic:

- Update scoring logic in ``correlation._refresh_host_inference``.
- Optionally add enrichment hints in ``enrichment.py`` and ``device_hints.json``.
- Verify role->node_role mapping in ``_refresh_topology_fields``.
- Add tests in ``tests/test_03_correlation_inference.py``.

Add a new event type:

- Add dataclass in ``backEnd/models/events.py``.
- Extend parsing in ``processors/extractors.py``.
- Handle event in ``processors/correlation.py``.
- Persist resulting model fields through entity/librarian if needed.
- Add tests for extractor + correlation path.

Change topology snapshot fields:

- Update ``frontEnd/supervisor_bridge.py`` projections and serializer.
- Update ``main_controller.normalizeHost/normalizeEdge``.
- Update rendering and details panel consumers.

Tune active scanning behavior:

- Update target selection in ``backEnd/main.py`` interval discovery section.
- Update nmap/ping strategy in ``active_discovery.py``.

- Validate safety constraints around private/public scope.

21. Extending the Frontend Safely

When adding UI controls:

- Add HTML element IDs in ``windows/main.html``.
- Bind in ``main_controller.js``.
- Keep renderer isolated from Node APIs (use preload only).
- Route backend-changing actions through IPC and bridge, not direct file writes from renderer.

When modifying canvas rendering:

- Keep hit-region bookkeeping consistent with rendered geometry.
- Preserve ``state.selectedHostId`` and drawer sync behavior.
- If adding mode-specific controls, gate pointer/keyboard behavior by ``state.mode``.

When modifying settings:

- Distinguish backend runtime settings vs frontend display settings.
- Backend settings must flow through ``updateSessionSettings`` IPC.

22. Security and Scan Safety

Design constraints:

- Active scanning is user-controlled and defaults conservative.
- Public/internet active scan permission is explicit and confirmation-gated.
- Private scope restriction is enforced in backend flow when ``allow_all_active_targets`` is false.

Operational guidelines:

- Never assume authorization to scan public or external ranges.
- Keep default configuration scoped to local/private network discovery.
- Treat role/topology output as inference, not absolute truth.

23. Known Gaps and Future Work

Current known gaps:

- No unified dependency lock file for backend Python packages.
- No formal migration/versioning layer for Mongo documents.
- Topology inference confidence is heuristic and may drift in complex networks.
- Passive capture behavior depends on platform tool availability and permissions.
- No CI pipeline is defined in repository for automated lint/test gates.

24. Contributor Checklist

Before submitting changes:

- Confirm startup + stop flow works from Electron.
- Validate no stale runtime flags persist unexpectedly.
- Run tests from ``tests/run_code_tests.py``.
- Verify topology snapshot still includes required frontend fields.
- If modifying inference or active scanning, test against both private-only and allow-all toggle states.
- Add or update tests when behavior changes.
- Keep process boundaries intact: renderer does not bypass preload/IPC.

25. Appendix: Quick Reference

Useful paths:

- ``frontEnd/main.js``
- ``frontEnd/supervisor_bridge.py``
- ``backEnd/main.py``
- ``backEnd/sensors/passive_listener.py``
- ``backEnd/sensors/active_discovery.py``
- ``backEnd/processors/correlation.py``
- ``runtime/session_config.json``
- ``runtime/session_update.json``

Useful commands:

- ``cd frontEnd && npm start``
- ``python tests/run_code_tests.py``